

Smart Meter Anomaly Detection Dashboard

Submitted in partial fulfillment of the requirements for the award of the degree of

Master of Computer Application (MCA)

By-

Student Name: **Devendra Singh Shakya**

Enrollment No: CUOL909961

Under the guidance of-

Guide/Mentor Name: **Vasanthi Chandran**

Certificate

Copy to the certificate received from Qollabb to be pasted here.

This page is intentionally kept available for the official certificate as required by the college format.

Declaration

I, **Devendra Singh Shakya**, hereby solemnly declare that the project report titled "Smart Meter Anomaly Detection Dashboard" submitted in partial fulfillment of the requirements for the award of the degree of Master of Computer Application (MCA) is my original work.

I further declare that:

- This project has been carried out by me during the academic year 2025-2026 under the supervision of **Vasanthi Chandran**.
- The work has not been submitted previously to any other university, institution, or examination body for the award of any degree, diploma, or certification.
- All sources of information used in this report have been duly acknowledged and referenced in accordance with academic ethics and plagiarism norms.
- The data presented in this report is authentic to the best of my knowledge and has not been fabricated or manipulated.
- I understand that if any part of this declaration is found to be false, my project may be rejected and disciplinary action may be taken as per institutional rules.

Place: Gurugram Date: 16/05/2026

Student Signature: Devendra Student Name: Devendra Singh Shakya

Enrollment No: CUOL909961

Acknowledgement

I express my sincere gratitude to my mentor, faculty members, department, institution, and supporting organization for their guidance and encouragement during the successful completion of this project. Their valuable suggestions helped me understand the practical importance of smart meter monitoring, anomaly detection, system analysis, implementation, and testing. I also thank my classmates, friends, and family members for their support throughout the development and documentation of this MCA project.

Abstract / Executive Summary

Smart electricity meters generate continuous readings such as voltage, current, and power. Monitoring these readings manually is difficult, especially when abnormal behavior such as theft-like low power usage, fault-like high consumption, or irregular meter activity occurs. The Smart Meter Anomaly Detection Dashboard project addresses this problem by building a real-time monitoring and anomaly detection system. The system uses historical smart meter readings as the baseline for normal behavior and applies two anomaly detection approaches: Isolation Forest for unusual individual readings and an LSTM-based sequence detector for time-series behavior. A live meter simulator generates readings for multiple devices and sends them to a FastAPI backend. The backend analyzes each reading, classifies it as NORMAL, THEFT, FAULT, or ANOMALY, calculates confidence, and exposes the results to a Flask dashboard. The dashboard visualizes live readings, anomaly counts, model decisions, recent events, and confidence values. The project demonstrates the integration of artificial intelligence with an IoT-style monitoring pipeline and covers data preparation, preprocessing, model training, live telemetry, inference, alert-oriented decision logic, visualization, and testing using simulated abnormal spikes.

Topic Of The Study

The topic of study is the application of artificial intelligence and anomaly detection in smart meter monitoring.

Objective

- To monitor smart meter readings and detect abnormal usage patterns in real time.

System Analysis

The system analyzes historical readings, runtime telemetry, anomaly model outputs, and dashboard requirements.

System Design

The design follows a modular pipeline consisting of data preparation, model training, backend inference, simulator, and dashboard layers.

System Implementation

The implementation uses Python, FastAPI, Flask, Chart.js, scikit-learn, NumPy, pandas, joblib, and pytest.

Testing

Testing is performed using automated pytest cases and simulated abnormal spikes.

Future Scope

Future scope includes real smart meter integration, database storage, cloud deployment, and SMS/email alerts.

Table of Contents

1. Chapter 1: Introduction
2. Chapter 2: System Study
3. Chapter 3: System Analysis
4. Chapter 4: System Design
5. Chapter 5: System Implementation
6. Chapter 6: Testing
7. Chapter 7: Results & Discussion
8. Chapter 8: Conclusion & Future Scope
9. Chapter 9: References
10. Chapter 10: Appendices

Chapter 1: Introduction

Electricity distribution systems increasingly depend on smart meters to record and transmit energy usage data. Smart meters provide information about consumption patterns, voltage variation, current usage, peak load, power faults, and suspicious usage behavior. When these readings are analyzed properly, they can support better energy management and faster fault identification.

The large volume of meter readings makes manual monitoring impractical. A human operator cannot continuously inspect every reading from every device and detect abnormal behavior at the exact time it occurs. Automated monitoring is therefore important in modern smart grid and utility management systems.

This project, Smart Meter Anomaly Detection Dashboard, implements a real-time monitoring system that learns normal behavior from historical readings and analyzes live readings generated by a simulator. The project combines machine learning, web APIs, dashboard visualization, and testing into one complete academic workflow.

The project focuses on three numerical features: voltage, current, and power. These values are enough to demonstrate how usage behavior can be monitored and how abnormal readings can be identified. The backend classifies readings as NORMAL or ANOMALY and further marks suspicious readings as THEFT, FAULT, or generic ANOMALY.

The system uses Isolation Forest and LSTM to provide two complementary anomaly detection approaches. Isolation Forest detects unusual individual readings, while LSTM observes recent power sequences. This combination helps explain both sudden abnormal spikes and sequence-based behavior.

1.1 Project Background

Smart meter monitoring is a useful application area for artificial intelligence because it produces continuous numerical data. Anomaly detection is suitable because abnormal situations may not always follow a fixed rule. A model trained on normal behavior can help detect unusual patterns more flexibly than static threshold checks.

1.2 Need Of The Project

- To reduce manual effort in meter monitoring.
- To identify theft-like and fault-like behavior quickly.
- To visualize meter readings in a user-friendly dashboard.
- To demonstrate AI integration in an IoT-style system.

Chapter 2: System Study

2.1 Existing System

Traditional meter monitoring systems are often based on periodic reading collection and manual review. In many cases, abnormal usage is identified only after billing analysis or customer complaints. This creates delay and may cause losses for utility providers.

Rule-based systems can identify simple abnormal values, but they are limited when behavior changes over time. Fixed thresholds may generate false alerts during peak usage and may miss unusual readings that do not cross a hard boundary.

Existing systems may also lack live dashboard visualization. Without visual summaries, it becomes difficult for users to understand whether alerts are isolated events or part of a larger pattern.

2.2 Proposed System

The proposed system is a real-time smart meter anomaly detection dashboard. It trains anomaly detection models using historical readings and simulated normal readings. It then receives live readings, applies model inference, classifies events, and displays results on a dashboard.

The system is modular. The training module prepares the models, the simulator acts as a monitoring agent, the backend performs inference, and the dashboard presents results. This makes the project easy to explain, test, and extend.

2.3 Feasibility Study

- Technical feasibility: The system uses widely available Python libraries and web frameworks.
- Operational feasibility: The dashboard is simple to run and understand.
- Economic feasibility: The project uses open-source tools and does not require paid services.

2.4 Requirement Study

Requirement	Description
Input	Voltage, current, power, device ID, timestamp, simulator label
Processing	Preprocessing, Isolation Forest scoring, LSTM sequence analysis
Output	Prediction, detected event, confidence, anomaly source
Users	Student, evaluator, faculty, utility monitoring demo user

Chapter 3: System Analysis

3.1 Problem Analysis

The problem is to detect abnormal smart meter behavior from continuous readings. The system must receive readings, compare them against learned normal behavior, classify the result, and present the output clearly.

3.2 Functional Requirements

- The system shall load and preprocess historical readings.
- The system shall train anomaly detection models.
- The system shall receive live readings through an API.
- The system shall classify readings as normal or anomalous.
- The system shall classify anomaly type as THEFT, FAULT, or ANOMALY.
- The system shall display live readings and recent events on a dashboard.
- The system shall support test evaluation through simulated labels.

3.3 Non-Functional Requirements

- Usability: Dashboard should be easy to understand.
- Maintainability: Modules should be separated by responsibility.
- Reliability: Backend should keep recent processed readings safely.
- Testability: Core logic should be covered by automated tests.

3.4 Data Flow

Historical data flows from `sample_data.csv` to `train_model.py`. Trained artifacts are saved in artifacts. Runtime readings flow from `meter_simulator.py` to `backend.py`. The Flask dashboard fetches processed readings and displays them in the browser.

Chapter 4: System Design

4.1 Architecture

sample_data.csv -> train_model.py -> artifacts -> meter_simulator.py -> backend.py -> flask_app.py -> templates/index.html

4.2 Module Design

Module	Responsibility
train_model.py	Loads data, trains Isolation Forest and LSTM, saves artifacts
anomaly_detection.py	Provides LSTM detector, utilities, metadata helpers
backend.py	Receives readings and performs model inference
meter_simulator.py	Generates live readings and abnormal spikes
flask_app.py	Serves dashboard and proxies backend data
templates/index.html	Shows chart, counters, events, and model details
run_project.py	Launches training, backend, dashboard, and simulator

4.3 Database Design

The current academic demo does not use a database. Recent readings are stored in backend memory for dashboard display. Future versions can store readings in SQLite, MySQL, PostgreSQL, or a time-series database.

4.4 Interface Design

The dashboard interface contains a live power timeline, summary counters, recent event cards, model status, anomaly source, confidence values, and latest decision display.

4.5 API Design

Method	Endpoint	Purpose
POST	/data	Receives smart meter reading and returns prediction status
GET	/data	Returns recent processed readings for dashboard display

Chapter 5: System Implementation

The project is implemented using Python. The training workflow is handled in `train_model.py`. It reads `sample_data.csv`, selects the main features, generates additional normal simulator-style readings, trains the Isolation Forest model, trains the LSTM detector, and saves model artifacts.

The `anomaly_detection.py` file contains the custom LSTM detector. The detector creates fixed-length sequence windows, scales values, trains a simple LSTM regressor, calculates prediction error, and learns an anomaly threshold from normal behavior.

The `backend.py` file implements the FastAPI application. It loads model artifacts, receives readings, calculates Isolation Forest and LSTM results, combines the model signals, classifies events, and stores recent enriched readings.

The `meter_simulator.py` file generates live readings for multiple meter devices. It also injects low-power and high-power abnormal readings to test theft-like and fault-like behavior.

The `flask_app.py` file serves the dashboard. The dashboard uses Chart.js to display a live power timeline and JavaScript polling to refresh results every two seconds.

5.1 Technology Used

Technology	Usage
Python	Main programming language
FastAPI	Backend inference API
Flask	Dashboard server
Chart.js	Graph visualization
scikit-learn	Isolation Forest and preprocessing pipeline
NumPy	Numerical computation
pandas	CSV and DataFrame handling
pytest	Automated testing

5.2 Installation And Execution

Install dependencies using: `pip install -r requirements.txt`

Run full project using: `python run_project.py`

Run tests using: `pytest -q`

Chapter 6: Testing

Testing ensures that the project works correctly and that model decision logic behaves as expected. The test suite is written using pytest and is located in the tests folder.

6.1 Testing Strategy

- Unit testing for helper functions.
- Backend route testing using FastAPI TestClient.
- Model decision testing using fake model objects.
- Simulator label comparison through match values.

6.2 Test Cases

Test Case	Expected Result
Feature vector creation	Voltage, current, and power are converted correctly
Normal reading during LSTM warmup	Prediction remains NORMAL
Low anomalous power	Detected as THEFT
High anomalous power	Detected as FAULT
Strong LSTM anomaly	Detected as ANOMALY
Separate meter history	Each meter has independent sequence history
API recent result limit	Only latest visible readings are retained

6.3 Validation

The simulator labels controlled abnormal spikes as THEFT or FAULT. The backend compares these labels with detected events using the match field. This helps evaluate correct detections, false positives, and precision-style behavior.

Chapter 7: Results & Discussion

The implemented system successfully demonstrates live anomaly detection for smart meter readings. Normal readings are displayed as normal events, while abnormal low-power and high-power spikes are highlighted as suspicious events.

The dashboard presents useful summaries such as total visible readings, combined anomaly count, detected theft-like events, detected fault-like events, Isolation Forest alerts, LSTM alerts, event match count, and latest decision.

The combined use of Isolation Forest and LSTM improves the explanatory value of the project. Isolation Forest detects sudden outliers, while LSTM evaluates recent sequence behavior. The dashboard also displays anomaly source, making the result easier to explain during viva and demonstration.

7.1 Discussion

The project is best understood as a complete monitoring workflow rather than only a machine learning experiment. It begins with historical data, prepares the data, trains models, receives live telemetry, performs inference, displays alerts, and supports testing.

Chapter 8: Conclusion & Future Scope

8.1 Conclusion

The Smart Meter Anomaly Detection Dashboard demonstrates how AI can be integrated with an IoT-style smart meter monitoring system. The project includes data preprocessing, anomaly detection, live simulation, backend APIs, dashboard visualization, alert-style classification, and automated testing. It is suitable for MCA academic submission because it combines machine learning, web development, system design, and practical implementation.

8.2 Future Scope

- Integration with real smart meter hardware.
- Long-term database storage for readings and alerts.
- SMS, email, or mobile push alert notifications.
- Cloud deployment with a production server.
- User login and role-based dashboard access.
- Automated retraining using new normal readings.
- Advanced anomaly models such as autoencoders or transformers.
- Daily, weekly, and monthly report generation.

Chapter 9: References

1. scikit-learn documentation, IsolationForest User Guide.
2. scikit-learn documentation, Pipeline and StandardScaler API references.
3. FastAPI official documentation, Path Operation and Request Body guides.
4. Flask official documentation, Quickstart and Template Rendering.
5. Chart.js official documentation, Line Chart and Tooltip configuration.
6. NumPy official documentation, ndarray and numerical operations.
7. pandas official documentation, CSV loading and DataFrame operations.
8. joblib official documentation, model serialization.
9. pytest official documentation, fixtures and test discovery.
10. Uvicorn official documentation, ASGI server usage.
11. Liu, F. T., Ting, K. M., and Zhou, Z. H., Isolation Forest, IEEE ICDM, 2008.
12. Hochreiter, S. and Schmidhuber, J., Long Short-Term Memory, Neural Computation, 1997.
13. Chandola, V., Banerjee, A., and Kumar, V., Anomaly Detection: A Survey, ACM Computing Surveys, 2009.
14. Aggarwal, C. C., Outlier Analysis, Springer.
15. Han, J., Kamber, M., and Pei, J., Data Mining: Concepts and Techniques.
16. Goodfellow, I., Bengio, Y., and Courville, A., Deep Learning, MIT Press.
17. Brownlee, J., Time Series Forecasting and sequence modelling concepts.
18. IEEE smart grid publications on advanced metering infrastructure.
19. NIST smart grid interoperability and cybersecurity guidance.
20. Python Software Foundation documentation, Python language reference.
21. Requests library documentation, HTTP client usage.
22. OpenAPI Specification documentation, REST API schema concepts.

Chapter 10: Appendices

10.1 Source Code Files

File	Description
README.md	Project overview and running instructions
sample_data.csv	Historical smart meter dataset
train_model.py	Model training script
anomaly_detection.py	LSTM anomaly detection utilities
backend.py	FastAPI inference backend
meter_simulator.py	Live reading simulator
flask_app.py	Flask dashboard server
templates/index.html	Dashboard user interface
run_project.py	One-command project launcher
tests/	Automated tests

10.2 User Manual

1. Install Python and required packages.
2. Run `python run_project.py` from the project folder.
3. Open `http://localhost:5000` in a browser.
4. Observe live readings, anomaly counts, and recent event cards.
5. Stop the services using `Ctrl+C` in the terminal.

10.3 Abbreviations

Term	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
IoT	Internet of Things
LSTM	Long Short-Term Memory

MCA	Master of Computer Applications
UI	User Interface